

Computational complexity

Brute Force

Recursivity

Backtracking

algorithmic study fundamentals

Computational complexity

Evaluating algorithms (in both time and space)

An intuitive approach to complexity

Search algorithm in a sorted array :

Target value T

Sorted array $A = \{A_1, A_2, A_3 \dots A_n\}$

Search for 47								
0	4	7	10	14	23	45	47	53

Goal: find an algorithm to search if T is present in A

An intuitive approach to complexity

Two algorithms:

Linear search algorithm

```
trouve = False
i = 0
while not trouve and i < len(A):
    if(A[i] == T):
        trouve = True
    i += 1

if trouve:
    print("trouvé")
else:
    print("pas trouvé")
```

Binary search algorithm

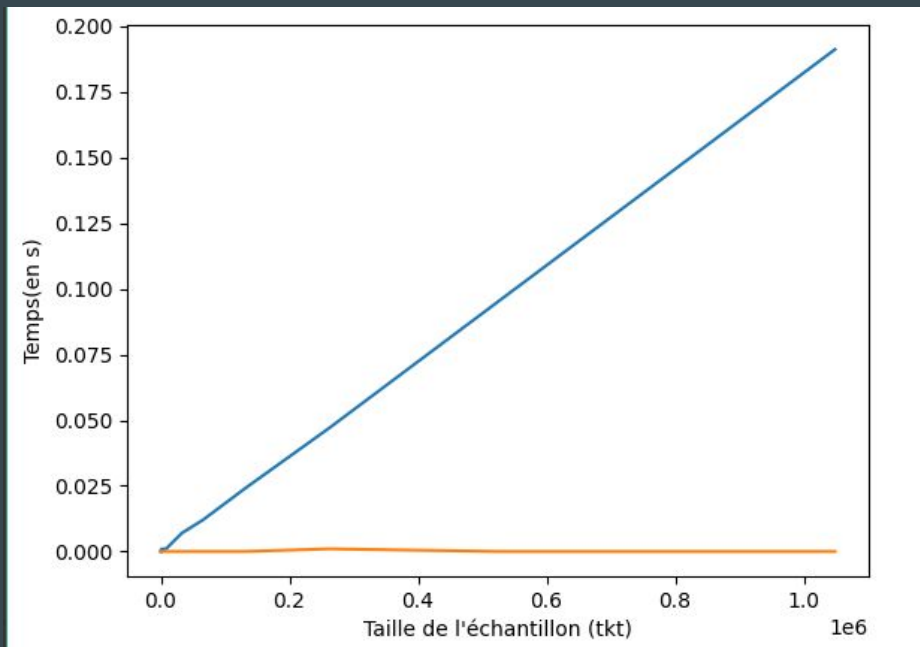
```
start = 0
end = len(A) - 1
resultat = "pas trouvé"
while(start <= end):
    mid = (start + end) // 2
    if(A[mid] > T):
        end = mid - 1
    elif(A[mid] < T):
        start = mid + 1
    else:
        resultat = "trouvé"

print(resultat)
```

How efficient are they?

An intuitive approach to complexity

Let's compare them with the module *time*:



In blue, the Linear search algorithm

In orange, the Binary search algorithm

What happened?

An intuitive approach to complexity

Linear search algorithm

```
trouve = False
i = 0
while not trouve and i < len(A):
    if(A[i] == T):
        trouve = True
    i += 1

if trouve:
    print("trouvé")
else:
    print("pas trouvé")
```

Loop of size n at worst, with fixed operations inside $\rightarrow$ complexity is called *linear* in n

Binary search algorithm

```
start = 0
end = len(A) - 1
resultat = "pas trouvé"
while(start <= end):
    mid = (start + end) // 2
    if(A[mid] > T):
        end = mid - 1
    elif(A[mid] < T):
        start = mid + 1
    else:
        resultat = "trouvé"

print(resultat)
```

Let's take a closer look at what happens with this algorithm...

An intuitive approach to complexity

Binary Search

	0	1	2	3	4	5	6	7	8	9
Search 23	2	5	8	12	16	23	38	56	72	91
	L=0	1	2	3	M=4	5	6	7	8	H=9
23 > 16 take 2 nd half	2	5	8	12	16	23	38	56	72	91
						L=5	6	M=7	8	H=9
23 > 56 take 1 st half	2	5	8	12	16	23	38	56	72	91
						L=5, M=5	H=6			
Found 23, Return 5	2	5	8	12	16	23	38	56	72	91



At each step the search interval is divided in half.
→ complexity is called *logarithmic* in $\log_2(n)$.

A more formal definition

- What we have evaluated previously is the *time complexity*, i.e the amount of time that it takes to run an algorithm, in function of its parameters
- When comparing memory consumption, we talk about *space complexity*

A more formal definition

When evaluating complexity, we use the *big O notation*:

$$f(x) = O(g(x))$$

if and only if there is x_0 and M such that:

$$|f(x)| \leq M g(x) \quad \text{for all } x > x_0$$

For example:

- $n^2 - 3n + 5 = O(n^2)$
- $42 = O(1)$
- $-7n \log_2 n = O(n \log n)$

This notation allows to evaluate how the program behaves depending on the size of the parameters.

Examples of time complexity

Let n be the size of an array.

Here are the time complexities of a few operations:

- $O(\log n)$ find an element in a sorted array with binary search
- $O(n)$ explore all elements of the array
- $O(n \log n)$ sort the array with a merge sort
- $O(n^2)$ find all the couples of elements of the array
- $O(n!)$ find all the permutations of the array

etc

Average VS worst case complexity

Sometimes the time needed to run an algorithms varies for inputs of the same size.

For example, the complexity of the quick sort is:

- $O(n \log n)$ on average
- $O(n^2)$ in the worst case

When talking about complexity, we usually refer to the *worst case* complexity.

Common complexities

- 1 constant
- $\log n$ logarithmic
- n linear
- $n \log n$ linearithmic
- n^2 quadratic
- n^3 cubic
- n^k polynomial
- k^n exponential
- $n!$ factorial

$O(1)$	 Oh, hell yes!
$O(\log n)$	 Ooh, nice.
$O(n)$	 Oh, cool.
$O(n^2)$	 Oh boy.
$O(2^n)$	 oh no oh no
$O(n!)$	 Oh, you're approaching me?

Equivalence time - complexity

total time = number of operations / operations per second

In python3 ...

total time = number of operations / $10^7 - 10^8$

Why is it useful?

Problem's constraints : $0 \leq N \leq 10^2$, $0 \leq C \leq 10^3$, Time limit = 1s

$N * C \Rightarrow 10^5$ operations $\Rightarrow$ 0.001s

$N^2 * C \Rightarrow 10^7$ operations $\Rightarrow$ 0.1s

$N * C^2 \Rightarrow 10^8$ operations $\Rightarrow$ 1s

$N! * C \Rightarrow \infty?$ operations $\Rightarrow$ NO

Some easily avoidable traps

```
array = ACollectionImplementation()  
targets = [1,2,5,6]  
for element in targets:  
    if(array.find(element)):    ← we don't know how this function is implemented  
        print(element,"was found")  
    else:  
        print(element,"was not found")
```

The Complexity of this algorithm is unknown at first glance, but often computable using documentation

Search space and brute force

If all you have is a hammer, everything looks like a nail

Search Space

- Ensemble of all solutions we have to choose from to get the optimal answer.
- Often what is asked from us is not the solution but one of its properties.

Search Space on an example : minimum pair

- Find the minimum pair on the list:

I.E. the pair with the minimum sum

0 5 8 6 -1 4 9 10 2



Search space : all the pairs

Make sense of your search space

Another representation:

Matrix where:

Line: first element of the pair

Column: second element

Implementation: two nested for loops

	0	5	8	6
0	-----	5	8	6
5	5	-----	13	11
8	8	13	-----	14
6	6	11	14	-----

Make sense of your search space

Another representation:

Matrix where:

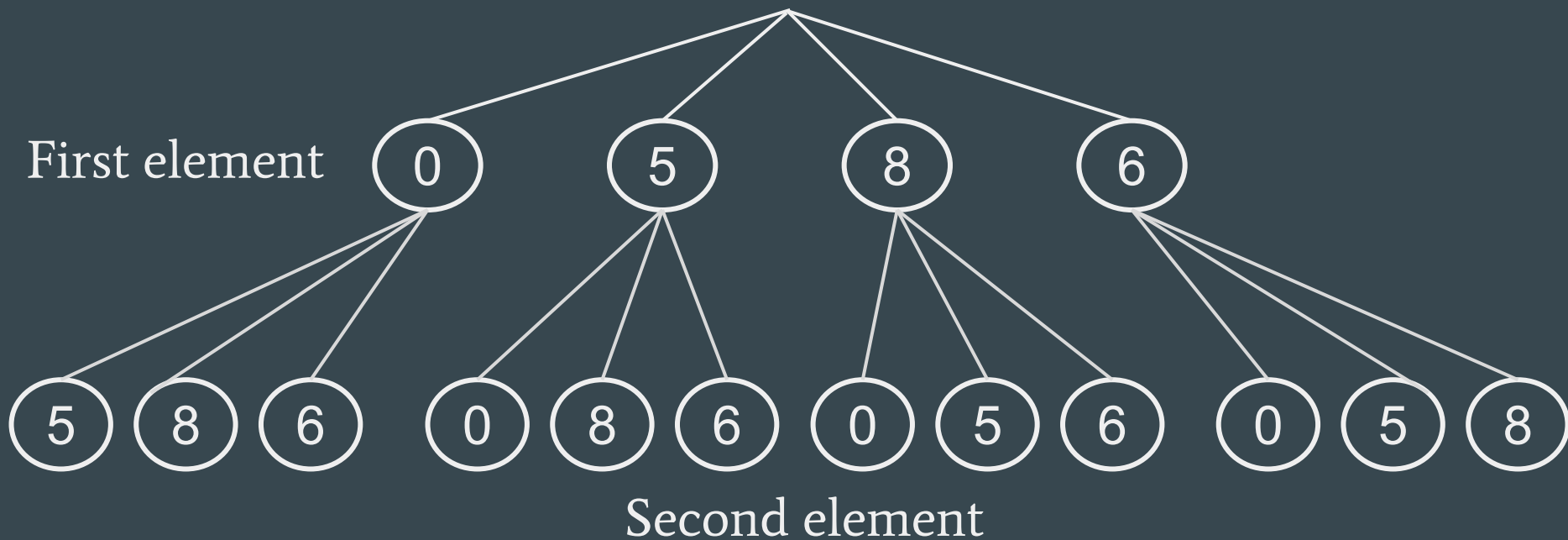
Line: first element of the pair

Column: second element

Implementation: two nested for loops

	0	5	8	6
0	---	5	8	6
5	5	---	13	11
8	8	13	---	14
6	6	11	14	---

Make sense of your search space



Brute force - introduction

Example: I want to find all subsets of a set which sum is 42.

I can:

- make a list of all subsets,
- for each subset, calculate the sum and check if it is 42.

Search space: all subsets of the input array

Test: the sum is 42

} A *brute force search* has a
search space and a *test*.

Brute force - construct the search space

The python module [itertools](#) provides useful tools to construct the search space.

```
>>> list(product("ab", "cd"))
[('a', 'c'), ('a', 'd'), ('b', 'c'), ('b', 'd')]
>>> list(permutations("abc", 2))
[('a', 'b'), ('a', 'c'), ('b', 'a'), ('b', 'c'), ('c', 'a'), ('c', 'b')]
>>> list(combinations("abc", 2))
[('a', 'b'), ('a', 'c'), ('b', 'c')]
>>> list(combinations_with_replacement("abc", 2))
[('a', 'a'), ('a', 'b'), ('a', 'c'), ('b', 'b'), ('b', 'c'), ('c', 'c')]
```

Note: we use `list()` because the return values are *lazy-evaluated* iterables.

→ these are very useful to save memory

Quick tour of theses functions

product($q_1, q_2, \dots, [\text{repeat}=1]$)

```
product('ABCD', repeat=2)
```

```
AA AB AC AD BA BB BC BD CA CB CC CD DA DB DC DD
```

Can take multiple iterables and output the cartesian product between them.

The repeat field indicate how many time do we take each iterable.

Complexity : $O((\prod q_i)^{\text{repeat}})$

With repeat=2, the complexity is $(q_1 * q_2)^2$

Where q_i is the size of each iterable.

Quick tour of theses functions

permutation(p, [r=len(p)])

```
permutations('ABCD', 2)
```

```
AB AC AD BA BC BD CA CB CD DA DB DC
```

Take an iterable and create every permutation of length r.

A permutation is an ordered (order is important) of element without repetition.

Complexity : $O\left(\frac{p!}{(p-r)!}\right)$ For example with r=2, the complexity is $p * (p - 1)$

Where p is the size of the iterable p.

Quick tour of these functions

combinations(p, r)

```
combinations('ABCD', 2)
```

```
AB AC AD BC BD CD
```

Take an iterable and create every combination of length r.

A combination is a non ordered of element without repetition, since it is non ordered, AB and BA are the same.

Complexity : $O\left(\binom{p}{r}\right) = O\left(\frac{p!}{r!(p-r)!}\right)$

Here, it's $\frac{p * (p-1)}{2}$

Where p is used as the size of the iterable p

Quick tour of theses functions

combinations_with_replacement(p, r)

```
combinations_with_replacement('ABCD', 2) | AA AB AC AD BB BC BD CC CD DD
```

Take an iterable and create every combination of length r with replacement.

Same as combination, but we can repeat the same element multiple times

Complexity : $O\left(\binom{p+r-1}{r}\right) = O\left(\frac{(p+r-1)!}{r!(p-1)!}\right)$ Here, it's $\frac{(p+1) * p}{2}$

Where p is used as the size of the iterable p

Brute force - let's solve our example

```
from itertools import combinations

arr = [int(n) for n in input().split()]

gen = (sub for size in range(len(arr))
        for sub in combinations(arr, size))

for sub in gen:
    if sum(sub) == 42:
        print(sub)
```

IN:

10 12 25 30 17 8 14 9 6

OUT:

(12, 30)

(25, 17)

(25, 8, 9)

(10, 12, 14, 6)

(10, 17, 9, 6)

gen is Python magic called a *generator expression*.

Ask us if you want to know more!

Why not use brute force ?

There is a problem called combinatorial explosion.

TL;DR: the search space is often a lot bigger than the input size (e.g. exponential).

Previous example: for an array of size n , there are $2^n - 1$ non-empty subsets...

For an algorithm in $O(n!)$ with $n=30$, your algorithm would still be running after the “end” of the universe.

$$10^{20} \text{ years} < \frac{30!}{365 * 24 * 3600 * 10^9}$$

Why ~~not~~ use brute force :)

- Brute force is a good way to **test** if more complicated algorithms are correct. (you compare their results on small sets)
- Sometimes you just don't know a better algorithm. (password decryption...)
- Sometimes, slow is “good enough” (will have plenty of time to run in practice)

But, can't we reduce the search space a little bit?

→ often you can use *backtracking*

Credits - Brute Force

Slides: Louis Sugy, Arthur Tondereau, William Michaud, Pierre Weisse

Backtracking

Stop it, before it's too late !

Backtracking - introduction

We need to reduce the *search space*

Example: sudoku

- Try to fill the sudoku
 - Everytime you encounter a conflict, change the last number
 - If no number is ok, erase and change the previous one
- etc

5	3	4	6	7	8	9	1	2
6	2	7	1	9	5	3	4	8
1	9	8	3	4	2	5	6	7
8	5	9	1	6				3
4			8		3			1
7				2				6
	6					2	8	
			4	1	9			5
				8			7	9

Backtracking - concepts

The idea of backtracking is to:

- build the elements of the search space incrementally
- eliminate wrong partial solutions → and therefore all solutions that contain them

You can think of the search space as a *tree*, you will often use a *recursive function*

E.g for sudoku, each level of the tree corresponds to a empty cell in the grid

→ the size of the search space is 9^n but many cases can be discarded

Implementation of a backtracking sudoku solver in Python

```
def is_valid(grid, i, j, val):
    line = grid[i]
    column = [grid[k][j] for k in range(9)]
    square = [grid[3 * (i // 3) + k][3 * (j // 3) + l]
               for k in range(3) for l in range(3)]
    return not (val in line or val in column or val in square)

def backtracking(grid, i, j):
    if i == 9: return True
    nexti, nextj = (i if j < 8 else i + 1), (j + 1) % 9
    if grid[i][j] != 0:
        return backtracking(grid, nexti, nextj)
    for val in range(1, 10):
        if is_valid(grid, i, j, val):
            grid[i][j] = val
            if backtracking(grid, nexti, nextj): return True
            grid[i][j] = 0
    return False
```

```
grid = [list(map(int, input().split()))
         for _ in range(9)]

if not backtracking(grid, 0, 0):
    print("Impossible")
else:
    print("\n".join(" ".join(
        map(str, grid[i]))
        for i in range(9)))
```

Implementation of a backtracking sudoku solver in Python

IN:

```
5 3 0 0 7 0 0 0 0
6 0 0 1 9 5 0 0 0
0 9 8 0 0 0 0 6 0
8 0 0 0 6 0 0 0 3
4 0 0 8 0 3 0 0 1
7 0 0 0 2 0 0 0 6
0 6 0 0 0 0 2 8 0
0 0 0 4 1 9 0 0 5
0 0 0 0 8 0 0 7 9
```

OUT:

```
5 3 4 6 7 8 9 1 2
6 7 2 1 9 5 3 4 8
1 9 8 3 4 2 5 6 7
8 5 9 7 6 1 4 2 3
4 2 6 8 5 3 7 9 1
7 1 3 9 2 4 8 5 6
9 6 1 5 3 7 2 8 4
2 8 7 4 1 9 6 3 5
3 4 5 2 8 6 1 7 9
```

Partial candidates explored: 6428

Total size of the workspace: $8,86293812 \times 10^{21}$

Credits - Backtracking

Slides: Louis Sugy, Arthur Tondereau

Sudoku sample: Wikipedia

Recursion

Who answers when you call yourself ?

But first, what is recursion ?

It is a method of solving problems that uses a function that calls itself from inside.

It looks something like that:

```
def myRecursiveFunction():  
    return myRecursiveFunction()  
  
print(myRecursiveFunction())
```

Warning : this code might show what recursion looks like, but it has several problems, can you spot it ?

"To understand recursion,
you must first understand
recursion"

14 year old programmer:



Oh shit, that's deep

So, what is wrong with it ?

```
def myRecursiveFunction():  
    return myRecursiveFunction()  
  
print(myRecursiveFunction())
```

1. The function has no parameters =>
 - i. The function calls an identical version of itself
 - ii. It is impossible to change the behaviour of the function
2. There are no base cases => what do we return when we hit the end of the loop ?

```
def myRecursiveFunction(n):  
    if n==0:  
        return 1  
    return myRecursiveFunction(n-1)*n  
  
print(myRecursiveFunction(9))
```


When to use it ?

- When the Main problem is divisible in similar smaller problems
- We could use it almost everywhere :

Iteration $\Leftrightarrow$ Recursion

So why use recursion at all ?

- To combine MULTIPLE sub-problems in order to get the final solution



Some examples

Fibonacci sequence : $F_0 = 0$ | $F_1 = 1$ | $F_n = F_{n-1} + F_{n-2}$

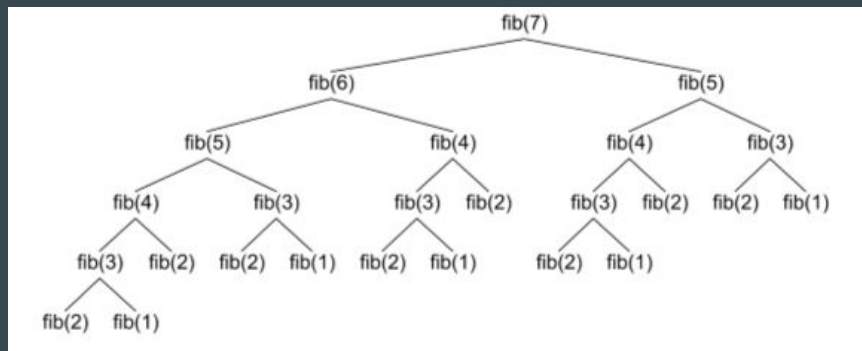
What we see: To know F_n we must first know the two terms before.

In summary, F_n is divisible in the same problems but smaller => Let's use recursion

```
def fibonacci(n):  
    if n==0: return 0  
    if n==1: return 1  
    return fibonacci(n-1)+fibonacci(n-2)  
  
print(fibonacci(7))
```

Works but be careful of the complexity !

Complexity : $O(2^n)$



Some examples

We have a family tree, and want to count the number of descendants of one person.

=> it is the sum of each child of this person and the number of descendants of each child

We can write $\text{numDesc}(\text{person}) = \text{numChild} + \text{sum}(\text{numDesc}(\text{child}) \text{ for each child})$

```
def numDesc(person):  
    children = person.children  
    numD = len(person.children)  
    for(child in children):  
        numD += numDesc(child)  
    return numD
```

How to make it ?

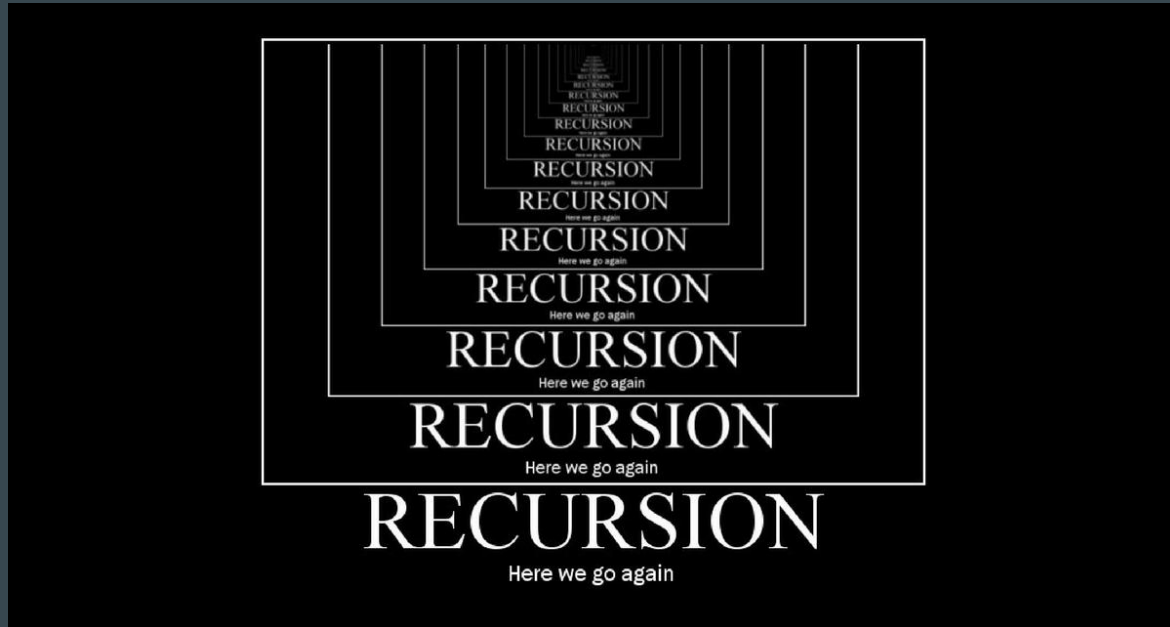
- Create a function (python syntax : `def myFunction(param):`)
- Divide the main problem in smaller problem (find the equation between problems)
- Identify how to stop the loop (base cases/no more parsing possible)
- Remember to return a result

```
def fibonacci(n):  
    if n==0: return 0  
    if n==1: return 1  
    return fibonacci(n-1)+fibonacci(n-2)  
  
print(fibonacci(7))
```

```
def numDesc(person):  
    childs = person.childs  
    numD = len(person.childs)  
    for child in person.childs:  
        numD += numDesc(child)  
    return numD
```

function
problem division
exit condition
return

Question ?



*Now, you're all set to
make your very first $O(n!)$
sorting algorithm :)*